

SD Basic - Alternate Key Indexes

Building an index on a field, selecting through it, and the three steps that report success while doing nothing useful.

An alternate key index lets you find records by the contents of a field instead of by record id. This page covers creating one, keeping it, and reading it from BASIC.

SD folds case, so a program may be written in either case. Keywords are shown here in lower case. In the tables, *italics* mark something you supply and **bold** marks a word typed as it stands; braces mark an optional part.

Creating an index takes three steps, and skipping any of them is silent

This is the whole of this page's warning, so it is first. Each step reports success on its own, and a program that stops after the first or second gets an index that exists and finds nothing.

```
create.index file.name field.name          ;* at the command line
```

then **close the file in every session that has it open**, then

```
build.index file.name field.name
```

then reopen the file.

WHAT EACH STEP ACTUALLY DOES

A dynamic file with three records, `TOWN` being field 2, holding `LONDON`, `LEEDS` and `LONDON`:

<code>create.index ZZAK TOWN</code>	<i>"Added index for TOWN",</i> <code>@system.return.code</code> 0
<code>indices(f)</code> on the already-open file variable	empty
<code>fileinfo(f, 13)</code> on it	0 — no indexes
after <code>close f</code> and reopening	<code>indices(f2)</code> is <code>TOWN</code> , <code>fileinfo(f2, 13)</code> is 1
<code>selectindex 'TOWN', 'LONDON'</code> at that point	0 ids

`build.index` with the file still open

"Cannot gain exclusive access to file",
`@system.return.code` **3021**

`build.index` after `close`

"Building index 'TOWN'... 3 records processed / Populating index...", `@system.return.code` **0**

`selectindex 'TOWN', 'LONDON'` after reopening

C1 and C3

An open file variable never learns about a new index. The handle carries the index list it had when it was opened. So a program that creates an index and then uses it in the same run finds nothing, and neither `indices()` nor `fileinfo()` gives it a hint — both report the state of the handle, not of the file. **Close and reopen.**

And `create.index` makes an empty index. It records the definition; it does not walk the existing records. Until `build.index` runs, the index is there, is reported by `indices()`, and matches nothing. A file that was indexed before it had data does not need building; one indexed afterwards always does.

`build.index` needs exclusive access and your own session counts. With the file open in the same program, it refuses with "Cannot gain exclusive access to file" and `@system.return.code` **3021**. **Test that return code** — the message goes to the capture variable, not to the screen, so a program that does not look at it carries on with an unbuilt index.

Once it is built, it maintains itself

After `build.index`, writing a fourth record with `TOWN` of `LONDON` made `selectindex 'TOWN', 'LONDON'` return **3** ids without any further action. Ordinary `write` and `delete` keep every index on the file up to date.

That is also the cost: each index makes every write a little more expensive. Index the fields you select on, not every field.

Selecting through an index

```
selectindex index.name {, value} from file.variable {to list}
```

With and without a value it returns completely different things, and neither form says so. On the file above:

CALL

WHAT COMES BACK

`selectindex 'TOWN', 'LONDON' from f to 4`

the record ids — **C1, C3**

```
selectindex 'TOWN' from f to 4
```

the distinct index values — LEEDS, LONDON

So the no-value form is how you ask "*what towns are there?*" and the with-value form is how you ask "*who is in London?*". A program that drops the value argument by accident gets a list of values where it expected ids, reads them as record ids, and finds nothing — with no error at any point.

Walking an index in order

```
setleft index.name from file.variable
setright index.name from file.variable
selectleft index.name from file.variable {setting variable} {to list}
selectright index.name from file.variable {setting variable} {to list}
```

`setleft` positions before the first index value, `setright` after the last. `selectright` then moves to the next value and selects its ids; `selectleft` moves back.

`setting` receives the index value the scan landed on. After `setleft 'TOWN' from f`, a `selectright` returns **1 id** with `setting` holding **LEEDS** — the first value in order.

These four take no `then` or `else` clause. Writing one is a compile error — "*Unrecognised statement*" — because the parser expects `setting` or `to` and finds a keyword it has no use for. To detect the end of the index, read the select list they build: an empty one means there was nothing further.

Walking with `selectright` is how you produce a report in index order without sorting, and how you page through a large file a value at a time rather than selecting all of it.

Asking what indexes a file has

```
indices(file.variable)
indices(file.variable, index.name)
```

one argument

the index names, as a dynamic array

two arguments

the definition of one index

`indices(f2, 'TOWN')` returns a **nine-field** definition whose field 2 is the **field number** the index is built on — `2` for a `TOWN` in field 2. Field 1 holds the dictionary-style definition the index was made from.

`fileinfo(file.variable, 13)` is the quick yes/no: `1` if the file has any index, `0` if not — subject to the open-handle caveat above.

Maintaining an index from BASIC

```
akread variable from file.variable, index.name, key then ... else ...
akwrite variable to file.variable, index.name, key
akdelete file.variable, index.name, key
akclear file.variable, index.name
akenable file.variable, index.name
akrelease file.variable, index.name
create.ak file.variable, index.name, ...
delete.ak file.variable, index.name
```

These reach into the index directly rather than letting SD maintain it. They exist for the case an ordinary index cannot express — an index whose key is computed by a program rather than taken from a field.

They are not the normal route and they will let you corrupt an index. Writing an index entry that does not match the record it points at produces a `selectindex` that returns ids whose records do not have the value you asked for. Nothing detects that except `build.index`, which rebuilds from the data and throws away whatever you put there. **Use the ordinary statements and let SD maintain the index unless you have a reason you can write down.**

`akenable` and `akrelease` turn maintenance off and on around a bulk load, which is the one common reason to touch these at all: loading a million records with three indexes live is far slower than loading them and building the indexes afterwards.

`listindex()`

```
listindex(list, delimiter, item)
```

Picks one entry out of a delimited list — it has nothing to do with alternate key indexes despite the name, and belongs with the list functions in SD Basic - Select Lists.

What is not here

Nothing in the index group has been removed from this port.

See also

[SD Basic - Select Lists](#) · [SD Basic - File Handling](#) · [SD Basic - Dynamic Arrays](#).
